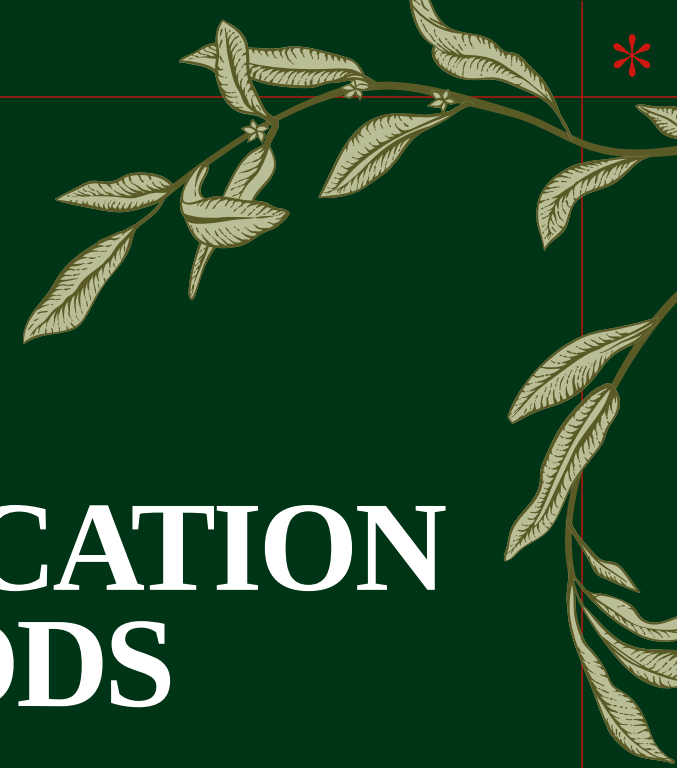


DEPLOYING PYTHON APPLICATION USING VARIOUS METHODS

SPANDANA BAINDLA



Agenda

- Using manual method
- Userdata
- Bash script
- Jenkins
- Terraform
- Terraform, Jenkins, git, github
- Docker
- K8s
- Pipeline



1. Build and deploy python applications in manual process

2-user@ip-172-31-90-116:~/flight-perdiction

```
from flask import Flask, render_template, url_for, request, redirect
import numpy as np
import pandas as pd
import joblib
import pickle

app = Flask(__name__)

# Load the model and the OneHot encoder
model = joblib.load('model.pkl')
ot = joblib.load('OneHot.pkl')

# Home route
@app.route('/')
@app.route('/home')
def main():
    return render_template('home.html')

# Predict route
@app.route('/predict', methods=['POST'])
def predict():
    int_features = [x for x in request.form.values()]
    c = ["Airline", "Source", "Destination", "Total_Stops", "month", "day"]
    df = pd.DataFrame(int_features, columns=c)
    l = onehot.transform(df.iloc[:, :4])
    c = onehot.get_feature_names_out()
    t = pd.DataFrame(l, columns=c)
    l2 = df.iloc[:, 4:]
    final = pd.concat([l2, t], axis=1)
    result = model.predict(final)
    print("The Result is :", result)

    print(int_features)

    return render_template("home.html", prediction_text="Flight Price Prediction")

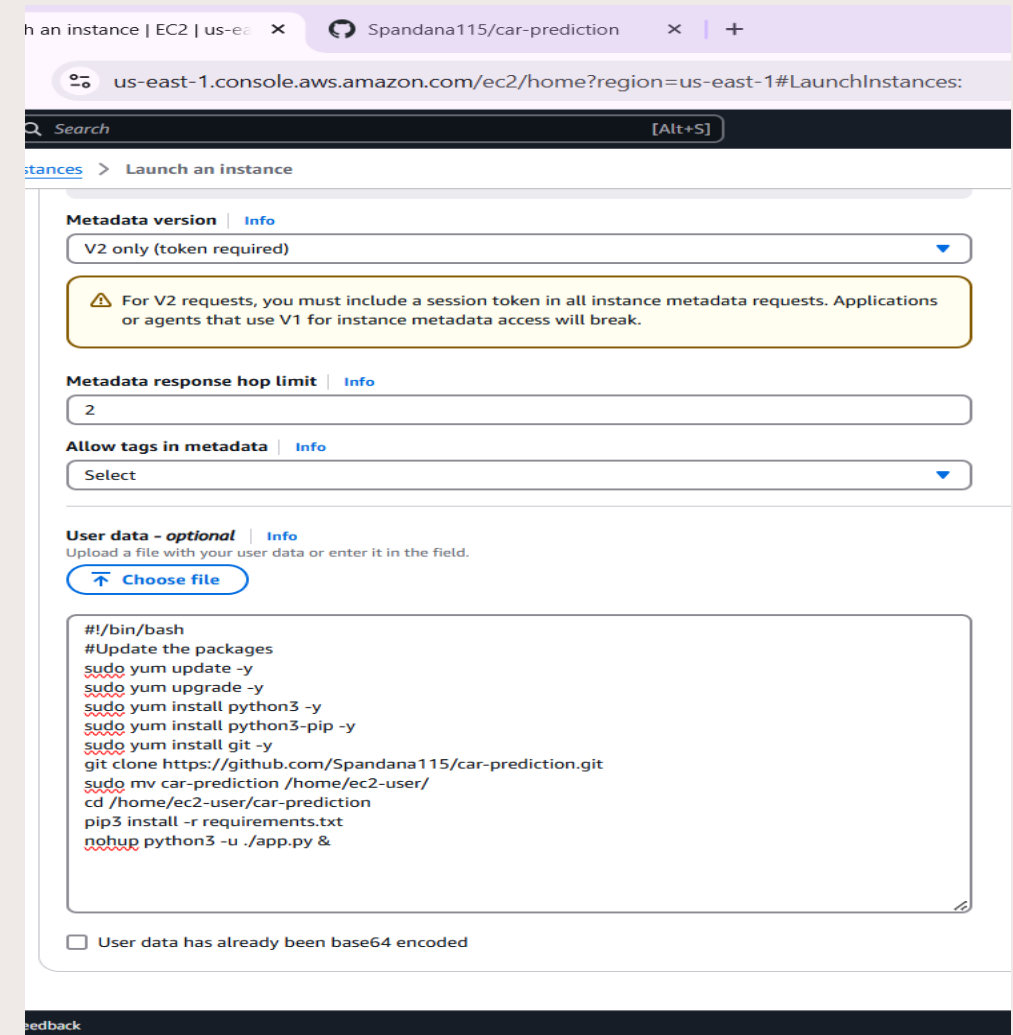
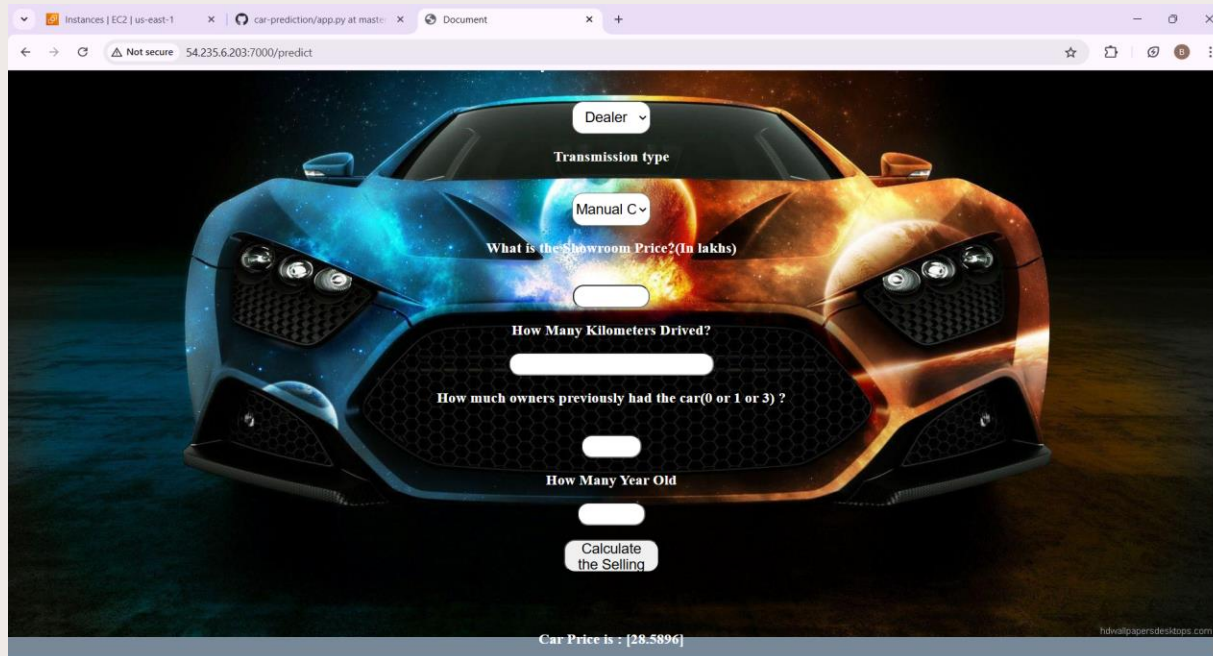
if __name__ == "__main__":
    app.run(host='0.0.0.0', port=9000, debug=True)
```

.py" 40L, 962B

- Install python and dependencies manually on the terminal where we are having application
- Clone the application's repository from the source control.
- Use pip3 install -r requirements.txt to install all required dependencies.
- Start the application by executing python3 app.py in the terminal.

2. Build and deploy python applications along with EC2 instance(userdata)

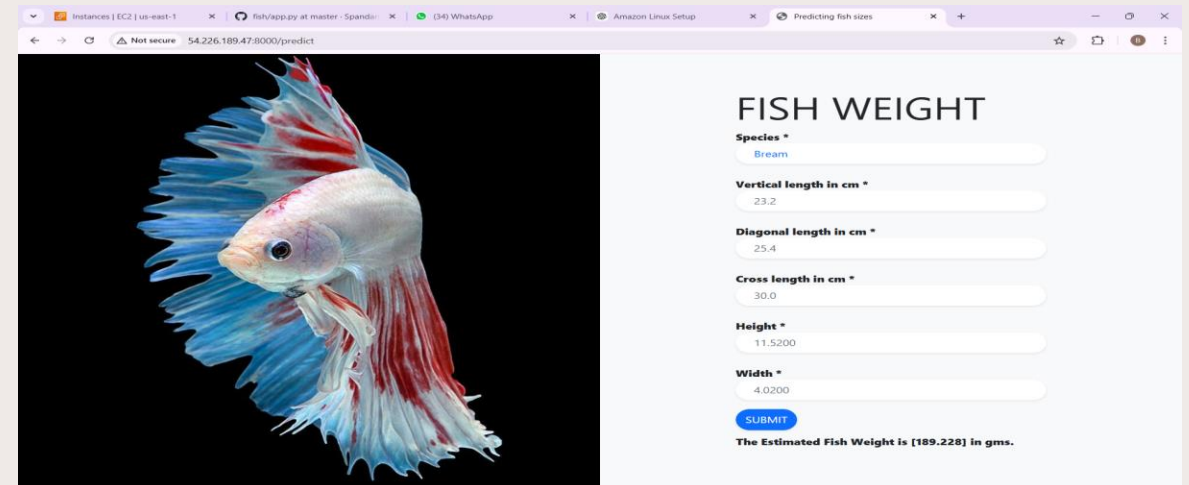
- Launch an EC2 instance.
- Provide a User Data script
- Application runs automatically on instance launch



3. Build and deploy python applications in Bash scripting

```
echo "Updating system packages..."
sudo yum update -y
sudo yum install python3 python3-pip git -y
sudo yum install git -y
echo "Cloning the repository..."
git clone https://github.com/Spandana115/fish.git
cd fish
echo "Installing dependencies..."
pip3 install -r requirements.txt
echo "Running tests..."
echo "Starting the application..."
nohup python3 -u app.py &
echo "Checking if the app is running..."
ps aux | grep 'python3 app.py'
```

- Write bash script file. data.sh to deploy python application.
- Using sh data.sh command we need to run the file.





4. Build and deploy python applications with Git, Github and Jenkins (execute shell)

- ❑ Launch a ec2 instance
- ❑ Setup Jenkins environment
- ❑ Create Jenkins job.
- ❑ Take the github code into Jenkins
- ❑ Run the commands in execute shell
- ❑ Build the job

```
C2 | us-east-1 | Build Deploy Python Jenkins | Spandana115/USA-Housing | Job-1 #2 Console [Jenkins] | USA Housing  
job/Job-1/2/console  
  
scipy-1.7.3 six-1.17.0 threadpoolctl-3.1.0 typing-extensions-4.7.1 zipp-3.15.0  
* Serving Flask app 'app'  
* Debug mode: on  
/var/lib/jenkins/.local/lib/python3.7/site-packages/sklearn/base.py:338: UserWarning: Trying to unpickle  
version 1.1.2 when using version 1.0.2. This might lead to breaking code or invalid results. Use at your  
own risk. For more info please refer to:  
https://scikit-learn.org/stable/modules/model\_persistence.html#security-maintainability-limitations  
UserWarning,  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on all addresses (0.0.0.0)  
* Running on http://127.0.0.1:8000  
* Running on http://172.31.17.37:8000  
Press CTRL+C to quit  
* Restarting with stat  
/var/lib/jenkins/.local/lib/python3.7/site-packages/sklearn/base.py:338: UserWarning: Trying to unpickle estimator LinearRegression from  
version 1.1.2 when using version 1.0.2. This might lead to breaking code or invalid results. Use at your own risk. For more info please refer  
to:  
https://scikit-learn.org/stable/modules/model\_persistence.html#security-maintainability-limitations  
UserWarning,  
* Debugger is active!  
* Debugger PIN: 567-014-121  
49.206.47.130 - - [23/Dec/2024 06:06:18] "GET / HTTP/1.1" 200 -  
49.206.47.130 - - [23/Dec/2024 06:06:20] "GET /favicon.ico HTTP/1.1" 404 -  
49.206.47.130 - - [23/Dec/2024 06:08:27] "POST /predict HTTP/1.1" 200 -
```

The screenshot shows the Jenkins 'Configure' page for a job named 'Job-1'. The left sidebar contains navigation links: General, Source Code Management, Build Triggers, Build Environment, Build Steps (selected), and Post-build Actions. The main area is titled 'Configure' and shows the 'Build Steps' section. A single step named 'Execute shell' is configured with the following command:

```
#!/bin/bash  
  
# Navigate to the workspace where the repository is cloned (Jenkins automatically checks out the code)  
cd $WORKSPACE  
  
# Install required dependencies from the requirements.txt file  
pip3 install -r requirements.txt  
  
# Run the Python application (app.py) on the specified host and port  
nohup python3 app.py > app.log 2>&1 &  
  
# Optionally, you can display logs or check the application status  
tail -f app.log
```

5. Build and deploy python applications with the Terraform(data.sh) and push the Terraform Scripted files in Github

- Create a instance and install terraform.
- Provide aws configurations to instance
- Create the terraform files for launching ec2 instance, vpc, route table, igw
- Initialize the terraform in the created files.
- Run the “terraform apply” to create the terraform resources.

```
ec2-user@ip-172-31-27-18:~  
vpc_id = aws_vpc.demovpc.id  
}  
  
resource "aws_route_table" "route" {  
  vpc_id = aws_vpc.demovpc.id  
  route {  
    cidr_block = "0.0.0.0/0"  
    gateway_id = aws_internet_gateway.demogateway.id  
  }  
  tags = {  
    Name = "Route to Internet"  
  }  
}  
  
resource "aws_route_table_association" "rt1" {  
  subnet_id      = aws_subnet.public_subnet1.id  
  route_table_id = aws_route_table.route.id  
}  
  
resource "aws_security_group" "demosg" {  
  vpc_id = aws_vpc.demovpc.id  
  
  ingress {  
    from_port = 80  
    to_port   = 80  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  ingress {  
    from_port = 443  
    to_port   = 443  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  ingress {  
    from_port = 22  
    to_port   = 22  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  ingress {  
    from_port = 8000  
    to_port   = 8000  
    protocol  = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  egress {  
    from_port = 0  
    to_port   = 0  
    protocol  = "-1"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
}  
"main.tf" 101L, 2065B
```



```
ec2-user@ip-172-31-27-18:~  
resource "aws_security_group" "demosg" {  
  vpc_id = aws_vpc.demovpc.id  
  
  ingress {  
    from_port = 80  
    to_port   = 80  
    protocol = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  ingress {  
    from_port = 443  
    to_port   = 443  
    protocol = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  ingress {  
    from_port = 22  
    to_port   = 22  
    protocol = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  ingress {  
    from_port = 8000  
    to_port   = 8000  
    protocol = "tcp"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  egress {  
    from_port = 0  
    to_port   = 0  
    protocol = "-1"  
    cidr_blocks = ["0.0.0.0/0"]  
  }  
  tags = {  
    Name = "Web SG"  
  }  
}  
  
resource "aws_instance" "demoinstance" {  
  ami           = "ami-0166fe664262f664c"  
  instance_type = "t2.micro"  
  key_name      = "spandana"  
  vpc_security_group_ids = [aws_security_group.demosg.id]  
  subnet_id     = aws_subnet.public_subnet1.id  
  associate_public_ip_address = true  
  user_data     = file("data.sh")  
  tags = {  
    Name = "My Public Instance"  
  }  
}
```

```
ec2-user@ip-172-31-27-18:~  
#!/bin/bash  
  
# Update the package lists  
sudo yum update -y  
  
# Install Python3, pip, and Git  
sudo yum install -y python3 python3-pip git  
  
# Clone the GitHub repository  
git clone https://github.com/Spandana115/Medical-Insurance.git  
  
# Move into the application directory  
cd Medical-Insurance  
  
# Install the required Python packages  
pip3 install -r requirements.txt  
  
# Start the Python application  
nohup python3 app.py &  
  
~  
~  
~  
~  
~  
~
```

Browser tabs: (22) WhatsApp, Instances | EC2 | us-east-1, Spandana | IAM | Global, ChatGPT, Editing Medical-Insurance, Document

Address bar: Not secure 54.175.172.247:8000/predict

Navigation: Home Contact Help

Know your Medical Insurance Charges



Sex

Smoker

Region

Age

BMI

Children

Expected Medical Charges in \$: [10034.99616366]

6. Build and deploy python applications with Git, github, Jenkins and Terraform

- Configure the Jenkins environment in the terminal.
- Create a new job in Jenkins and add the AWS Credentials Plugin.
- Set up AWS credentials under "Manage Jenkins" using Global Credentials.
- Include the installation commands for Terraform, AWS CLI, Python, and pip in the "Execute Shell" section of the Jenkins job.
- Trigger the Jenkins job by clicking on "Build Now."
- Verify the build process and ensure the application is accessible.

7. Build and deploy python applications with Docker

- Launch an EC2 instance and establish a connection using the terminal.
- Install Docker and start the Docker service on the instance.
- Create a Dockerfile to package and deploy the application.
- Build the Docker image with the command: `docker build -t my-python-app .`
- Run the container using: `docker run -p 8000:8000 my-python-app`.
- Access the application through the instance's public IP and specified port (e.g., `http://<public-ip>:8000`).
- Push the built image to Docker Hub for centralized access.

8. Build and deploy python applications with Docker and k8's

- Set up a KOPS cluster on the instance.
- Create two clusters: one for the control plane and one for nodes.
- Develop a deployment YAML file referencing the Docker image for the Python application.
- Create a service YAML file to expose the application.
- Access the application via the service's Load Balancer URL.

9. Build and deploy python applications with the Git, github, Jenkins and Terraform (with Build periodically, pollscm and webhooks)?

1. Set up Jenkins and create a new Jenkins job.
2. Configure "Build Periodically" to schedule builds at defined intervals using a cron expression.
3. Add commands in the "Execute Shell" section and enable the "Build Periodically" option.
4. Enable "Poll SCM" to trigger builds whenever there are changes in the source code repository.
5. Set up webhooks in the GitHub repository for real-time build triggers.
6. Test the Jenkins builds using all three triggering methods: Scheduled builds, SCM polling, and GitHub webhooks.

10. Build and deploy python applications with pipeline method (create clone job and Build job) with Build periodically, pollscm and webhooks?

- Set up a Pipeline Job in Jenkins.
- Define a pipeline with three stages:
 - Code: Handle source code checkout or retrieval.
 - Dependency: Install required dependencies.
 - Deploy: Deploy the application to the target environment.
- Use pipeline syntax to configure triggering methods:
- Set up webhooks in the GitHub repository to trigger builds in real time.
- Test and verify builds using the configured triggering methods and ensure each stage executes successfully.

THANK YOU ☺

